



Trailhead



[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Create Lightning Web Components

Create Lightning Web Components

Learning Objectives

After completing this unit, you'll be able to:

- Describe the contents of each component file.
- Create JavaScript methods for a Lightning web component.
- Use Lifecycle Hooks in the component lifecycle.

Play Time

Say you want to build a data display element, independent of a specific object in Salesforce. A good example is the [productCard component](https://github.com/trailheadapps/ebikes-lwc/tree/master/force-app/main/default/lwc/productCard) (<https://github.com/trailheadapps/ebikes-lwc/tree/master/force-app/main/default/lwc/productCard>) in the ebikes sample repo. Let's examine that card component, and build our own version from scratch so you can see how it evolves into a full-fledged Lightning web component. You'll quickly get the basics when you build up the parts of a component and explore more samples.

Step up to an Org

In this unit, we develop a Lightning web component using Visual Studio Code with the Salesforce extension.

What You Need

As stated in the first unit, you need some familiarity with Salesforce DX to continue. To complete this unit, you need:



Hi NELLURI, ask me about products, features, and pricing, or connect to a sale...



Summarize this unit

What is an Agentblazer?

Quiz me!



A Look Into the HTML File

Lightning web component HTML files all include the `<template>` tag. The `<template>` tag contains the HTML that defines the structure of your component. Let's look at the HTML for a simplified version of the `productCard` component from the `ebikes` repo. <https://www.lightningdesignsystem.com/2e1ef8501/p/253f7e-name-value-list> documentation.



This badge is focused on getting Lightning Web Components working, not on best practice for page and data layout in an HTML page. Here is a link to the [Salesforce Lightning Design System 2 \(SLDS\) Named Value List](https://www.lightningdesignsystem.com/2e1ef8501/p/253f7e-name-value-list) (<https://www.lightningdesignsystem.com/2e1ef8501/p/253f7e-name-value-list>) documentation.

Follow along by pasting these examples in VS Code.

1. Create a project by selecting **SFDX: Create Project** from the Command Palette in VS Code. Accept the standard template and give it the project name `bikeCard`.
2. Under `force-app/main/default`, right-click the `lwc` folder and select **SFDX: Create Lightning Web Component**.
3. Enter `app` for the name of the new component.
4. Press **Enter** and then press **Enter** again to accept the default `force-app/main/default/lwc`.
5. Paste the following into `app.html` (replacing any existing HTML in the file).

```

1  <template>
2    <div>
3      <div>Name: {name}</div>
4      <div>Description: {description}</div>
5      <div>Category: {category}</div>
6      <div>Material: {material}</div>
7      <div>Price: {price}</div>
8      <div><img src={pictureUrl} alt={name}/></div>
9    </div>
10 </template>

```





```
5     category = 'Mountain';
```

```
6     material = 'Steel';
```

```
7     price = '$2,700';
```

[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

```
8     pictureUrl = 'https://s3-us-west-2.amazonaws.com/dev-or-  
9     devrl-s3-bucket/sample-apps/ebikes/electrax4.jpg';
```

Create Lightning Web Components

7. Save the files.

Now let's play with a real-world example. Say you want to display data, but you know it can take some time to load. You don't want the user wondering what's up. You can use `lwc:if` [copy](#) and `lwc:else` [copy](#) conditional directives within your template to determine which visual elements are rendered.

1. Paste the following into **app.html**. The content in the “display” `div` [copy](#) tag doesn't appear until the value of `ready` [copy](#) is `true` [copy](#) in the HTML file.

```
1  <template>  
2    <template lwc:if={ready}>  
3      <div id="display">  
4        <div>Name: {name}</div>  
5        <div>Description: {description}</div>  
6        <div>Category: {category}</div>  
7        <div>Material: {material}</div>  
8        <div>Price: {price}</div>  
9        <div><img src={pictureUrl} alt={name}/></div>  
10     </div>  
11   </template>  
12   <template lwc:else>  
13     <div id="waiting">Loading...</div>  
14   </template>  
15 </template>
```



2. Paste the following into **app.js**. This holds our data values and sets a 3-second timer. After 3 seconds, the content should appear. (Of course, this is only for testing





```

10     ready = false;
11     connectedCallback() {
12         onTitled();
13         trailmix creator id=srebello7&trailmix slug=salesforce-platform-developer);
14     }, 3000);
15 }

```

[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Create Lightning Web Components

3. Save the files.

Base Lightning Web Components

Now, you don't want to build all your components from the ground up. So let's explore using a base Lightning web component. And of course, there are lots of components, including field types, display controllers, navigation items, and more. All of them are listed in the [Component Reference \(https://developer.salesforce.com/docs/component-library/overview/components\)](https://developer.salesforce.com/docs/component-library/overview/components).

Let's make the details of the bike stand out. In the **app.html** file, replace the div tags for material and category in the last example with a lightning-badge component. Here's the HTML.

```

1  <template>
2      <template lwc:if={ready}>
3          <div id="display">
4              <div>Name: {name}</div>
5              <div>Description: {description}</div>
6              <lightning-badge label={material}></lightning-badge>
7              <lightning-badge label={category}></lightning-badge>
8              <div>Price: {price}</div>
9              <div><img src={pictureUrl} alt={name}/></div>
10         </div>
11     </template>
12     <template lwc:else>

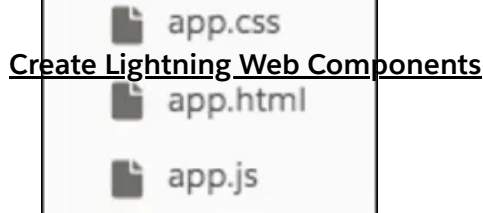
```





automatically linked by name and location.

[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics/create-lightning-web-components?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



All Lightning web components have a namespace that's separated from the folder name by a hyphen. For example, the markup for the Lightning web component with the folder name `app` in the default namespace `c` is `<c-app>` [copy](#).

However, the Salesforce platform doesn't allow hyphens in the component folder or file names. What if a component's name has more than one word, like “mycomponent”? You can't name the folder and files `my-component`, but we do have a handy solution.

Use camel case to name your component `myComponent` [copy](#). Camel case component folder names map to kebab-case in markup. In markup, to reference a component with the folder name `myComponent`, use `<c-my-component>` [copy](#).

For example, the LWC Samples repo has the [viewSource folder](https://github.com/trailheadapps/lwc-recipes/tree/master/force-app/main/default/lwc/viewSource) (<https://github.com/trailheadapps/lwc-recipes/tree/master/force-app/main/default/lwc/viewSource>) containing the files for the `viewSource` component. When the `hello` component [references the viewSource component in HTML](https://github.com/trailheadapps/lwc-recipes/blob/master/force-app/main/default/lwc/hello/hello.html) (<https://github.com/trailheadapps/lwc-recipes/blob/master/force-app/main/default/lwc/hello/hello.html>), it uses `c-view-source` [copy](#).

OK. Let's look at the JavaScript.

Working with JavaScript

Here's where you make stuff happen. As you've seen so far, JavaScript methods define what to do with input, data, events, changes to state, and more to make your component work.





The `export` `copy`  statement defines a class that extends the `LightningElement` `copy`  class. As a best practice, the name of the class usually matches the file name of the Lightning Web Components Basics (/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) JavaScript class, but it's not a requirement.

~~Create Lightning Web Components~~ The LWC Module

Lightning Web Components uses modules (built-in modules were introduced in ECMAScript 6) to bundle core functionality and make it accessible to the JavaScript in your component file. The core module for Lightning web components is `lwc` `copy` .

Begin the module with the `import` `copy`  statement and specify the functionality of the module that your component uses.

The `import` `copy`  statement indicates the JavaScript uses the `LightningElement` `copy`  functionality from the `lwc` `copy`  module.

```
1  // import module elements
2  import { LightningElement } from 'lwc';
3  // declare class to expose the component
4  export default class App extends LightningElement {
5      ready = false;
6      // use lifecycle hook
7      connectedCallback() {
8          setTimeout(() => {
9              this.ready = true;
10             }, 3000);
11      }
12  }
```



- `LightningElement` `copy`  is the base class for Lightning web components, which allows us to use `connectedCallback()` `copy` .
- The `connectedCallback()` `copy`  method is one of our lifecycle hooks. You'll learn more about lifecycle hooks in the next section. For now, know that the method is triggered when a component is inserted in the document object model (DOM). In





• Created

- Added to the DOM

- Rendered in the browser

[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=sfello7&trailmix_slug=salesforce-platform-developer\)](#)

- Encountering errors

- Removed from the DOM

Create Lightning Web Components

Respond to any of these lifecycle events using callback methods. For example, the

`connectedCallback()` [copy](#) is invoked when a component is inserted into the DOM. The

`disconnectedCallback()` [copy](#) is invoked when a component is removed from the DOM.

In the JavaScript file we used to test our conditional rendering, we used the

`connectedCallback()` [copy](#) method to automatically execute code when the component is inserted into the DOM. The code waits 3 seconds, then sets `ready` [copy](#) to `true`

[copy](#) .

```
1 import { LightningElement } from 'lwc';
2 export default class App extends LightningElement {
3   ready = false;
4   connectedCallback() {
5     setTimeout(() => {
6       this.ready = true;
7     }, 3000);
8   }
9 }
```



When you use this example in an editor like VS Code, you might see a

lint warning "Restricted async operation...." for the `setTimeout()` [copy](#)

function. This warning indicates you're using an async operation that's often misused; it delays behavior based on time instead of waiting for an event. In this case, `setTimeout()` [copy](#) is suitable to demonstrate an arbitrary time delay, and the warning should not prevent you from using it.





Decorators

Decorators are often used in JavaScript to modify the behavior of a property or function. [Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=s...&trailmix_creator_id=s...&trailmix_creator_id=s...&trailmix_creator_id=s...&trailmix_creator_id=s...\)](#) [Trailmix creator id: s...&trailmix_creator_id=s...&trailmix_creator_id=s...&trailmix_creator_id=s...&trailmix_creator_id=s...](#) place it before the property or function.

Create Lightning Web Components

```
1 import { LightningElement, api } from 'lwc';  
2 export default class MyComponent extends LightningElement {  
3   @api message;  
4 }
```



You can import multiple decorators, but a single property or function can have only one decorator. For example, a property can't have `@api` `copy` and `@wire` `copy` decorators.

Examples of Lightning Web Components decorators include:

- **@api:** Marks a field as public. Public properties define the API for a component. An owner component that uses the component in its HTML markup can access the component's public properties. All public properties are reactive, which means that the framework observes the property for changes. When the property's value changes, the framework reacts and rerenders the component.



Field and property are almost interchangeable terms. A component author declares fields in a JavaScript class. An instance of the class has properties. To component consumers, fields are properties. In a Lightning web component, only fields that a component author decorates with `@api` `copy` are publicly available to consumers as object properties.

- **@track:** Tells the framework to observe changes to the properties of an object or to the elements of an array. If a change occurs, the framework rerenders the component. All fields are reactive. If the value of a field changes and the field is used in a template—or in the getter of a property used in a template—the framework



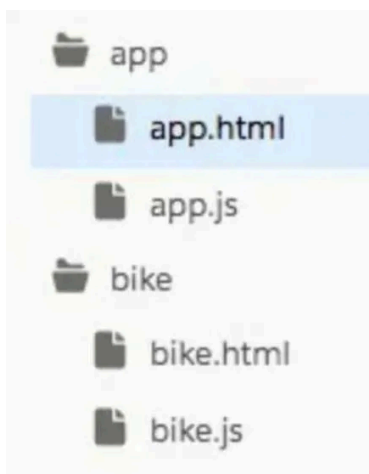


array. Some legacy examples may still use `@track`  where it isn't needed, but that's OK because using the decorator doesn't

[Lightning Web Components Basics \(Content Learn Modules Lightning Web Components Basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics/create-lightning-web-components?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Create Lightning Web Components : Gives you an easy way to get and bind data from a Salesforce org.

Here's an example using the `@api`  decorator to render a value from one component (bike) in another component (app). The file structure looks like this:



The app component uses the following HTML.

```
1 <!-- app.html -->
2 <template>
3   <div>
4     <c-bike bike={bike}></c-bike>
5   </div>
6 </template>
```



The app component uses the following JavaScript.

```
1 // app.js
2 import { LightningElement } from 'lwc';
3 export default class App extends LightningElement {
```





```
1 <!-- bike.html -->
```

[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

```
4 <p>{bike.name}</p>
```

[Create Lightning Web Components](#)



The bike component uses the following JavaScript.

```
1 // bike.js
2 import { LightningElement, api } from 'lwc';
3 export default class Bike extends LightningElement {
4   @api bike;
5 }
```



We're moving fast and you've been able to work with some code in VS Code. In the next unit, we will deploy some code and talk more about the environment where the components live.

Resources

- [Lightning Web Components Developer Guide: Reactivity](https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.reactivity) (<https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.reactivity>).
- [Lightning Web Components Developer Guide: Reference](https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.reference) (<https://developer.salesforce.com/docs/component-library/documentation/lwc/lwc.reference>) (includes HTML Template Directives, Decorators, and more)
- [MDN web docs: this](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this) (<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this>).
- [Salesforce Lightning Design System 2 \(SLDS\): Named Value List](https://www.lightningdesignsystem.com/2e1ef8501/p/253f7e-name-value-list) (<https://www.lightningdesignsystem.com/2e1ef8501/p/253f7e-name-value-list>).





[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/lightning-web-components-basics/create-lightning-web-components?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Create Lightning Web Components

- ☒ **B** The template tag is our version of a div tag.
- ☐ **C** The template tag replaces standard components.
- ☐ **D** The template tag contains the HTML for your component.
- ☐ **E** The template tag inserts pre-defined elements.

2 What do you have to include in a component's JavaScript file?

- ☐ **A** @api
- ☐ **B** An import statement with the LightningElement wrapper and an export statement with your class name
- ☐ **C** A template tag
- ☐ **D** An import statement with a file path and an export statement with your class name





Second attempt earns 50 points. Three or more earns 25 points.
[Lightning Web Components Basics \(/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/lightning-web-components-basics?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Check the Quiz to Earn 100 Points

Create Lightning Web Components

Learn

Badges
Trails
Trailmixes
Superbadges
Trailhead Academy
Career Paths

Certifications

Certifications
Maintain Certifications
Verify Certifications

Community

Trailblazer Community
Events
Agentblazers
Salesforce Developers
Salesforce Admins
Code of Conduct
Report Illegal Content (EU)

Extras

Trailhead Login
Sales Enablement
Trailblazer Store
Salesforce Help

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

Your Privacy Choices

Engli

